AdaptRouter: Continual Learning to Route across Evolving Tasks and Models

Jingpu Yang[1,2] Fengxian Ji[3] Tianhao Li[3]
Yufeng Wang[1] Juanfan Wu[5] Shuxin Zheng[2] Xiaoqing Zhang[2,4*]

[1]Beihang University [2]Beijing Zhongguancun Academy
[3]Northeastern University [4]Renmin University of China
[5]Beijing Institute of Technology

Abstract

Model routing assigns each query to a suitable foundation model, but both the tasks and the available models keep changing, and existing routers are either untrained for the pool at hand or fit once and forget earlier tasks when updated. We present AdaptRouter, which casts routing as continual policy learning: a compact multimodal decision model reads each question with cards describing the current experts and learns from native task feedback by reinforcement learning, without routing labels, to select one frozen expert. New tasks are learned without replay, new experts enter as cards, and a quality-first advantage provably keeps cost from overriding quality. Learning five benchmarks in four stages, AdaptRouter loses at most 0.4 points on earlier tasks, whereas feedback-trained routers lose up to 7.0; as the pool grows from three to ten experts it uses the added ones, and exploring 5% of the questions comes within 1.9 held-out points of budgets up to 90%. On a pool of five API models, it exceeds GPT-6.1 Sol and GPT-6 Astra (low) by over 22 points in weighted average at under a third of their cost.

1 Introduction

Foundation models and the tasks they serve keep changing: new models are released, existing ones are updated, repriced, or retired, and the questions that users bring shift over time. Model routing assigns each question to a suitable model in a pool (Lu et al., 2024; Li et al., 2026; Ma et al., 2026) and exploits large differences in capability and price, which make the most expensive configuration often unnecessary. On our open-weight suite, enabling reasoning in Qwen3.5-9B multiplies its parameter-weighted cost by about six but raises its average accuracy by only 1.5 points (Table 1); on our API suite, GPT-6 Astra at low reasoning effort

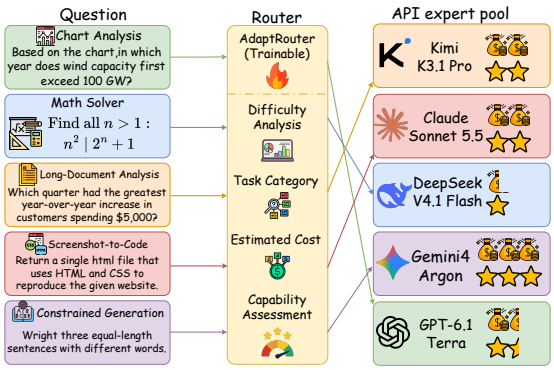


Figure 1: **Task-aware routing with AdaptRouter.** The router selects one expert per question from task requirements, expert capabilities, and cost.

costs five times as much as GPT-6.1 Sol for a gain of 0.3 points (Table 3). Which expert suits which question, however, depends on the task mix and the pool, and both keep moving: a router fit once becomes stale, and refitting it on all past data after every change is costly, so routing has to be learned continually.

We therefore study continual routing, in which a router is updated as new tasks arrive and the expert pool changes while remaining accurate and economical, which raises three requirements. First, the router must learn each new task from limited feedback while preserving what it has learned, without revisiting earlier data. Second, it must adapt to a changing pool: experts should enter as inputs rather than as fixed outputs, and a new pool should be learnable from a small exploration budget without routing labels. Third, every update must keep correctness first: the router has to anticipate from the input alone, including its images, which experts will succeed, and no cost trade-off may let cost outweigh correctness in the learning signal.

Existing routers meet these requirements only in

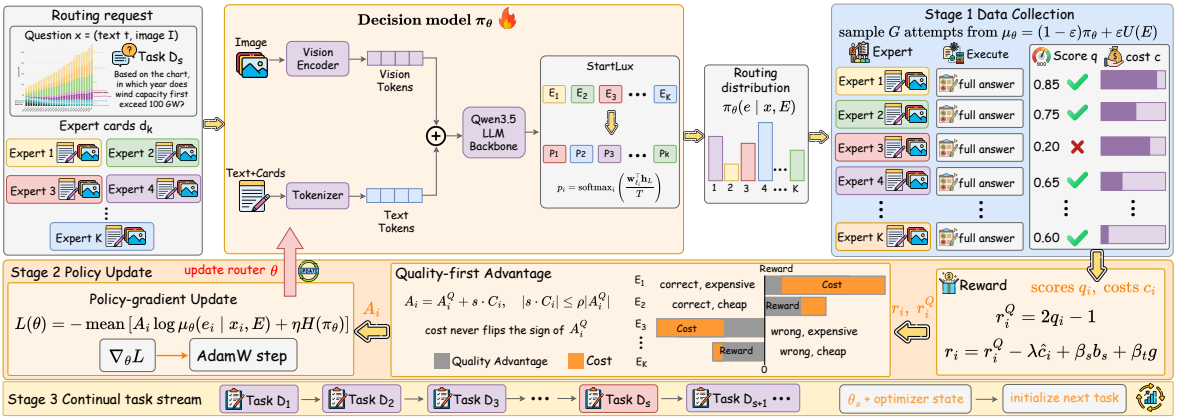


Figure 2: **Overview of AdaptRouter.** A decision model maps a question and expert cards to a policy π_θ; sampled experts are scored by quality and cost, and a quality-first advantage updates π_θ. Each stage continues from the previous policy, and a new pool changes only the cards.

part. Released and prompted routers (Tran et al., 2025; StartLux, 2026; SimpleJev, 2026; Strands Labs, 2026) select an expert from model descriptions or user-defined domains, so they accept a new pool, but they do not learn from its feedback. Feedback-trained routers (Ong et al., 2025; Zhuang et al., 2024; Feng et al., 2025; Li et al., 2012; Zhou et al., 2026a) learn to predict expert quality, mostly from frozen query embeddings with dedicated parameters per expert, and are fit once on a fixed split. Updated on new tasks without replay, such routers forget earlier ones, losing up to 7.0 points in our experiments (Table 2). Reinforcement-learning routers (Zhang et al., 2025a) train a language model to coordinate several calls on text-only tasks and fold cost into the reward as a weighted term, so a large cost weight can outweigh correctness.

We present AdaptRouter, which casts routing as continual policy learning (Fig. 2). As illustrated in Fig. 1, AdaptRouter feeds the complete question, including any images, together with short cards describing the current experts to a compact multimodal decision model, which already defines a distribution over listed candidates and selects one frozen expert to produce the entire answer. The policy learns from the native scores of its own sampled decisions, without routing labels, so the same procedure serves a new task that has an automatic scorer and a new pool described by cards. Three components address the three requirements: (i) continual training that carries the full optimizer state across stages and learns only from newly introduced data, without replay, regularization, or stage-specific heads; (ii) card-conditioned routing, in which a changed pool requires new cards and prices but no new head; and (iii) a quality-first learning signal that combines an engine-calibrated cost term with a leave-one-out group advantage in which cost ranks equally good experts but never reverses a quality preference. We use StartLux-0.8B as the decision model and report a pointer-head alternative in Appendix A.

Our contributions are:

- **Continual routing framework.** AdaptRouter trains compact multimodal decision models as card-conditioned routing policies from native feedback, learning new tasks without replay and new pools without architectural change, while provably keeping quality ahead of cost.
- **Adaptation to evolving tasks and pools.** Across four task stages without replay, AdaptRouter loses at most 0.4 points on earlier tasks, whereas feedback-trained routers lose up to 7.0. As the pool grows from three to ten experts, accuracy rises from 66.7% to 75.5%, and exploring 5% of the questions comes within 1.9 held-out points of budgets up to 90%.
- **Accuracy at low cost.** On 15 text and vision benchmarks, AdaptRouter improves a released 0.8B decision model by up to 21 points and reaches 97% of a 9B reasoning expert's accuracy at 12.6% of its cost. On five API models, it has the highest weighted average of all compared methods, over 22 points above GPT-6.1 Sol and GPT-6 Astra (low) at under a third of Sol's cost.

2 Related Work

Model routing. Early systems fuse model outputs (Jiang et al., 2023), cascade models until an answer is accepted (Chen et al., 2023; Yue et al., 2024), or choose between a small and a large model (Ding et al., 2024). Learned routers predict expert quality from reward models (Lu et al., 2024), preference labels (Ong et al., 2025), query and model embeddings (Chen et al., 2024c; Zhuang et al., 2024), graphs (Feng et al., 2025), or bandit feedback (Li et al., 2012); released and prompted routers select from domains or candidate lists without task-specific training (Tran et al., 2025; StartLux, 2026; SimpleJev, 2026; Strands Labs, 2026), and Router-R1 (Zhang et al., 2025a) learns multi-call routing with reinforcement learning. Benchmarks quantify routing gains (Hu et al., 2024; Li et al., 2026; Ma et al., 2026), and recent work frames routing as quality minus priced cost (Dekoninck et al., 2025; Zhou et al., 2026a; Yu et al., 2026). AdaptRouter instead learns a card-conditioned multimodal policy continually from native feedback, enforcing quality priority within every training group.

Continual and reinforcement learning. Continual learning mitigates forgetting through regularization (Kirkpatrick et al., 2017; Li and Hoiem, 2017) or replay (Rebuffi et al., 2017; Rolnick et al., 2019), including for multimodal instruction tuning (He et al., 2023; Chen et al., 2024a); adaptive routers keep past outcomes in external memory (Zhou et al., 2026b), while feedback-trained routers are typically fit once (Zhou et al., 2026a). Reinforcement learning with verifiable rewards improves reasoning in language and vision–language models (Shao et al., 2024; DeepSeek-AI et al., 2026; Liu et al., 2025; Shen et al., 2025) by optimizing generated answers. AdaptRouter instead optimizes one discrete decision per question with REINFORCE (Williams, 1992) and a leave-one-out baseline (Ahmadian et al., 2024), retaining earlier tasks without regularization, replay, or memory.

Efficient multimodal inference. Other methods reduce the cost of a single model by pruning visual tokens (Chen et al., 2024b; Shang et al., 2026; Zhang et al., 2025c; Yang et al., 2026; Wen et al., 2025), sparse experts (Shazeer et al., 2017; Lin et al., 2024), dynamic depth (Raposo et al., 2024; Han et al., 2021), or adaptive thinking (Fang et al., 2025; Zhang et al., 2025b); AdaptRouter selects among unmodified models and is complementary to them.

3 Problem Formulation

A router is updated over stages $s = 1, \dots, S$; at stage s, a dataset $\mathcal{D}_s$ of new tasks arrives, and the router chooses among a pool $\mathcal{E}_s$ of frozen experts that may change as models are added, retired, or repriced. Each expert e_k is described by a short card d_k (name, input modalities, reasoning mode).

Routing objective. For a question $x = (t, \mathcal{I})$ with text t and possibly empty images $\mathcal{I}$, the router $\pi_\theta(e \mid x, \mathcal{E}_s)$ commits to one expert, whose complete answer has native quality $q(x, e) \in [0, 1]$ and cost $c(x, e) \geq 0$. At stage s, it maximizes

$$J_s(\theta) = \mathbb{E}_{x \sim \mathcal{D}_s} \, \mathbb{E}_{e \sim \pi_\theta(\cdot \mid x, \mathcal{E}_s)} \Big[\big(2q(x, e) - 1\big) - \lambda \frac{c(x, e)}{\bar{c}} \Big], \quad (1)$$

where $\bar{c}$ is the mean cost of a reference expert, so λ trades $\lambda/2$ of accuracy for one reference execution (Dekoninck et al., 2025; Zhou et al., 2026a). No routing labels are given.

Continual protocol. At stage s, the router starts from θ_{s-1} and trains *only* on $\mathcal{D}_s$; after each stage it is tested on all datasets so far, measuring retention on earlier ones and plasticity on the new one. A new pool changes only the cards and prices, not the parameters. We study new tasks under a fixed pool (Section 5.2) and new pools of different size and composition (Sections 5.3 and 5.5).

Cost for comparison. Training uses the engine-calibrated cost of Section 4.2; for comparing open-weight experts, we report a parameter-weighted token cost (Pope et al., 2023; Srivatsa et al., 2024)

$$C = \textstyle\sum_x N(e_x)\big(n_{\text{in}}(x, e_x) + n_{\text{out}}(x, e_x)\big), \quad (2)$$

where $N(e_x)$ is the selected expert's size in billions and $n_{\text{in}}, n_{\text{out}}$ count its input and generated tokens; API experts use token cost at list prices.

4 Method

AdaptRouter trains a compact multimodal decision model as the routing policy π_θ and keeps training it as tasks and pools change (Fig. 2). The model reads the question and the expert cards (Section 4.1); sampled experts are scored by native quality and cost (Section 4.2), and a quality-first advantage updates the policy (Section 4.3). The same procedure runs at every stage: parameters and optimizer state carry over, each stage trains only on its new questions

Table 1: Benchmark scores (%). Avg.: request-weighted; Cost: relative to Qwen3.5-9B (thinking).

Method	LLMRouterBench					MMR-Bench V2						RobustRouteBench				Avg.	Cost
	Acc. ↑ MathBench	Acc. ↑ KOR-Bench	Acc. ↑ SimpleQA	Acc. ↑ FinQA	Acc. ↑ MELD	Acc. ↑ RealWorldQA	Acc. ↑ OCRBench	Acc. ↑ CharXiv	Acc. ↑ MathVerse	Acc. ↑ BLINK	aAcc. ↑ HallusionBench	Acc. ↑ HellaSwag	HSR ↑ FollowBench-zh	Acc. ↑ C-Eval	Acc. ↑ AI2D		
I. Direct and random baselines																	
Qwen3.5-9B (non-thinking)	95.60	57.60	6.60	37.60	60.60	71.86	83.56	60.90	72.03	61.40	73.55	89.20	46.41	82.80	82.40	65.81	16.81%
Qwen3.5-9B (thinking)	96.60	74.00	6.40	35.80	59.80	68.46	82.11	57.40	69.48	61.80	74.95	90.60	63.21	85.20	82.20	67.30	100.00%
Random	88.80	46.00	3.00	28.80	48.80	24.08	31.89	24.00	28.74	25.60	32.46	75.60	41.54	74.20	33.20	38.39	9.44%
II. Released and prompted routing methods																	
StartLux 0.8B (Original)	84.80	40.20	3.00	24.60	45.00	31.02	47.22	44.40	35.89	51.80	54.71	65.40	36.79	67.40	42.00	44.09	12.03%
JevAny-4B (Original)	86.20	49.00	6.20	27.80	51.40	61.91	72.33	52.10	41.89	55.80	61.12	80.00	45.00	71.40	65.60	55.24	16.90%
Arch (Image-to-Text)	81.00	27.00	0.40	5.20	0.00	3.01	22.67	3.10	1.79	3.40	9.42	21.00	34.36	67.60	7.60	17.92	5.42%
Qwen3.5-4B (Prompted Selector)	90.00	50.60	3.40	28.80	42.00	57.72	58.89	32.60	39.21	41.20	52.10	78.80	44.23	71.20	43.60	48.31	8.64%
Strands Qwen3.5-2B (Decider)	84.20	37.80	2.40	20.20	42.40	54.84	24.56	21.80	28.99	58.00	45.29	72.00	34.36	63.20	37.00	39.72	6.42%
III. Feedback-trained routing methods																	
Embedding-kNN	93.80	53.80	4.00	27.20	59.80	21.99	79.89	58.40	73.69	23.20	72.55	89.40	48.08	75.40	85.00	57.91	11.86%
MLP Reward Predictor	96.00	58.00	6.60	32.00	60.20	42.93	82.78	60.00	73.05	30.40	74.15	89.40	46.28	80.40	85.80	61.46	12.72%
EmbedLLM (adapted)	95.40	58.00	6.00	34.60	60.00	43.98	83.22	60.00	73.44	42.60	73.75	89.40	46.92	79.20	86.20	62.33	12.74%
LinUCB	96.20	53.00	5.80	31.00	60.20	57.72	82.78	60.10	74.84	40.20	73.35	88.80	45.51	80.80	84.20	62.81	12.62%
LatentRouter (adapted)	95.80	57.00	4.00	23.80	60.20	10.21	83.00	60.60	72.54	1.80	74.75	89.00	47.05	81.20	83.80	56.60	12.65%
IV. RL-trained routing methods																	
StartLux 0.8B (Full FT, ours)	96.20	58.40	5.80	37.60	60.20	69.24	83.00	57.70	72.67	61.20	72.55	89.20	46.41	81.00	82.00	65.08	12.56%
StartLux 0.8B (LoRA, ours)	94.80	59.00	5.20	36.40	51.00	65.45	80.78	56.10	69.09	62.20	71.94	84.40	45.00	80.00	82.40	63.04	8.81%

Table 2: Continual routing scores (%). Stage-4 parentheses: change from each dataset's first stage; Cost as in Table 1.

Method	Stage 1		Stage 2			Stage 3				Stage 4 (change vs. first)					S4 cost
	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ Human Eval+	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ Human Eval+	Acc. ↑ IF Eval	
I. Direct and random baselines															
Qwen3.5-9B (non-thinking)	92.60	85.88	92.60	85.88	74.40	92.60	85.88	74.40	84.15	92.60 (+0.00)	85.88 (+0.00)	74.40 (+0.00)	84.15 (+0.00)	85.40	20.15%
Qwen3.5-9B (thinking)	73.40	86.08	73.40	86.08	77.60	73.40	86.08	77.60	89.02	73.40 (+0.00)	86.08 (+0.00)	77.60 (+0.00)	89.02 (+0.00)	91.87	100.00%
Random	53.60	14.88	53.60	14.88	58.20	53.60	14.88	58.20	68.29	53.60 (+0.00)	14.88 (+0.00)	58.20 (+0.00)	68.29 (+0.00)	77.26	13.09%
II. Released and prompted routing methods															
StartLux 0.8B (Original)	39.80	21.44	39.80	21.44	42.40	39.80	21.44	42.40	61.59	39.80 (+0.00)	21.44 (+0.00)	42.40 (+0.00)	61.59 (+0.00)	75.60	20.63%
JevAny-4B (Original)	42.20	31.00	42.20	31.00	51.80	42.20	31.00	51.80	73.78	42.20 (+0.00)	31.00 (+0.00)	51.80 (+0.00)	73.78 (+0.00)	84.10	24.97%
Arch (Image-to-Text)	30.80	5.84	30.80	5.84	33.00	30.80	5.84	33.00	42.07	30.80 (+0.00)	5.84 (+0.00)	33.00 (+0.00)	42.07 (+0.00)	12.01	13.37%
Qwen3.5-4B (Prompted Selector)	53.20	20.52	53.20	20.52	62.60	53.20	20.52	62.60	66.46	53.20 (+0.00)	20.52 (+0.00)	62.60 (+0.00)	66.46 (+0.00)	82.26	14.37%
Strands Qwen3.5-2B (Decider)	59.60	12.60	59.60	12.60	47.00	59.60	12.60	47.00	56.10	59.60 (+0.00)	12.60 (+0.00)	47.00 (+0.00)	56.10 (+0.00)	72.27	9.28%
III. Feedback-trained routing methods															
Embedding-kNN	73.60	38.68	74.20	39.08	66.60	75.20	39.20	67.20	85.37	74.40 (+0.80)	38.92 (+0.24)	68.00 (+1.40)	85.37 (+0.00)	84.47	12.71%
MLP Reward Predictor	75.80	37.84	77.00	38.48	72.20	77.40	38.60	74.20	86.59	71.00 (-4.80)	37.96 (+0.12)	75.20 (+3.00)	85.98 (-0.61)	85.40	17.70%
EmbedLLM (adapted)	74.80	38.28	75.60	38.48	75.60	76.40	38.56	75.60	88.41	77.20 (+2.40)	38.48 (+0.20)	75.00 (-0.60)	89.63 (+1.22)	82.81	15.71%
LinUCB	76.20	38.80	74.00	38.08	67.80	73.60	37.96	67.80	84.15	71.80 (-4.40)	38.12 (-0.68)	68.80 (+1.00)	82.93 (-1.22)	82.26	9.37%
LatentRouter (adapted)	64.00	37.68	63.40	37.64	43.80	63.40	36.96	43.80	51.83	57.00 (-7.00)	37.56 (-0.12)	75.20 (+31.40)	77.44 (+25.61)	85.21	18.84%
IV. RL-trained routing methods															
StartLux 0.8B (Full FT, ours)	92.20	85.84	91.80	85.44	74.00	92.00	85.52	73.80	84.10	92.20 (+0.00)	86.02 (+0.18)	74.40 (+0.40)	84.32 (+0.22)	84.66	16.83%
StartLux 0.8B (LoRA, ours)	84.60	83.38	89.40	84.59	71.80	90.60	85.52	73.40	78.66	90.80 (+6.20)	85.68 (+2.30)	73.00 (+1.20)	78.66 (+0.00)	83.36	14.77%

with no replay, regularizer, or stage-specific head, and a new pool changes only the cards and prices.

4.1 Decision Models as Card-conditioned Routers

Routing request. The router receives the question text, its original images (encoded by the router's own vision encoder), and the expert cards in a random order per question, together with the objective stated in natural language. The cards are the only place where the pool enters the router: adding, retiring, or replacing an expert changes the cards, not the parameters.

Policy. Our primary router is the released *StartLux-0.8B* (StartLux, 2026) on a Qwen3.5 backbone (Qwen Team, 2026), which scores candidates by the next-token logits of their letter labels; we train it with full finetuning or LoRA (Hu et al., 2021), and a JevAny-style pointer head (SimpleJev, 2026) also works (Appendix A). At test time, only $\hat{e}(x) = \arg\max_e \pi_\theta(e \mid x, \mathcal{E})$ is executed, after a single prefill of the router.

4.2 Quality-first Cost-aware Reward

Engine-calibrated cost. Each execution's cost is estimated from its token counts with per-expert rates measured once on an inference engine (Kwon et al., 2023):

$$c(x,e) = a_e + p_e n_{\text{in}} + p'_e n_{\text{in}}^2 + v_e n_{\text{vis}} + \sum_{k=1}^{n_{\text{out}}-1} (o_e + o'_e(n_{\text{in}} + k)), \quad (3)$$

covering request overhead, prefill, vision encoding, and decoding over a growing context; costs are normalized as $\hat{c} = c/\bar{c}$.

Reward. For an attempt that routes x to e and obtains native quality q, the reward is

$$r(x,e) = \underbrace{(2q-1)}_{r^Q} - \lambda\,\hat{c}(x,e) + \beta_s\,b_s(x,e) + \beta_t\,g(x,e). \quad (4)$$

The first two terms implement Eq. (1); the training-only shaping terms credit a correct answer from a small non-reasoning expert (b_s) and the margin by which the most expensive reasoning expert beats every cheaper expert (g), computed from all experts' scores on the training question. We set λ so that the cost spread of the pool is not negligible relative to the quality gap of 2; otherwise the policy collapses to the most accurate expert.

4.3 Policy Optimization

Quality-first group advantage. For each training question, we draw G independent attempts from the exploration policy $\mu_\theta = (1-\epsilon)\,\pi_\theta + \epsilon\,\mathcal{U}(\mathcal{E})$, execute each selected expert once, and score its complete answer. Let r_i and r_i^Q denote the full reward and the quality term of attempt i in a group. Leave-one-out baselines (Ahmadian et al., 2024) yield the quality advantage and the full-reward advantage,

$$A_i^Q = r_i^Q - \tfrac{1}{G-1}\textstyle\sum_{j\neq i} r_j^Q, \qquad A_i^R = r_i - \tfrac{1}{G-1}\textstyle\sum_{j\neq i} r_j, \quad (5)$$

and $C_i = A_i^R - A_i^Q$ is the component contributed by cost and shaping. We combine them as $A_i = A_i^Q + s\,C_i$, where, if the qualities within the group differ,

$$s = \min\Big(1,\ \min_{i:\,C_i\neq 0} \frac{\rho\,|A_i^Q|}{|C_i|}\Big), \qquad \rho \in (0,1), \quad (6)$$

and $s = 1$ if all qualities are equal or all $C_i = 0$. Groups in which no attempt obtains positive quality receive zero advantage.

Proposition 1 (Quality priority) *For every attempt i with $A_i^Q \neq 0$, $\operatorname{sign}(A_i) = \operatorname{sign}(A_i^Q)$ and $|A_i - A_i^Q| \le \rho\,|A_i^Q|$.*

Proof. If all qualities are equal, every $A_i^Q = 0$ and the claim is vacuous. Otherwise, Eq. (6) gives $s|C_i| \le \rho|A_i^Q|$ for every i with $C_i \neq 0$; the inequality also holds when $C_i = 0$. Thus $|A_i - A_i^Q| = s|C_i| \le \rho|A_i^Q| < |A_i^Q|$ whenever $A_i^Q \neq 0$, so A_i has the same sign as A_i^Q.

Cost and shaping thus only rank attempts of equal quality, whereas the composite advantage A_i^R ($s = 1$) can reverse quality preferences. The router minimizes the policy-gradient loss (Williams, 1992)

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{B}|}\sum_{i\in\mathcal{B}}\Big[A_i \log \mu_\theta(e_i \mid x_i,\mathcal{E}) + \eta\,\mathcal{H}\big(\pi_\theta(\cdot \mid x_i,\mathcal{E})\big)\Big] \quad (7)$$

with one AdamW step per batch; the entropy bonus keeps the policy from saturating on a single expert. Because cost never reverses the sign of a quality advantage (Proposition 1), every continual stage stays anchored to correctness.

5 Experiments

We evaluate the three requirements of continual routing in turn: learning new tasks without forgetting (Section 5.2), adapting to new expert pools (Section 5.3), and accurate, low-cost routing on the main suite and on a new pool of API models (Sections 5.4 and 5.5).

5.1 Setup

Expert pools. NT and T denote non-reasoning and reasoning modes (outputs capped at 4,096/16,384 tokens). M3 contains Qwen3.5-2B (NT), 4B (NT), and 9B (T) (Qwen Team, 2026); M5 adds 4B (T) and 9B (NT); M10 adds Gemma-4-12B, MiniCPM-V-4.5 (NT), GLM-4.6V-Flash, Step3-VL-10B, and Ovis2.5-9B (T). The API pool contains GPT-6.1 Terra, DeepSeek V4.1 Flash, Claude Sonnet 5.5, Gemini 4 Argon, and Kimi K3.1 Pro; GPT-6.1 Sol and GPT-6 Astra serve as references.

Benchmarks. The continual sequence adds MATH-500 and ChartQA, MMLU-Pro, HumanEval+, and IFEval (Lightman et al., 2023; Masry et al., 2022; Wang et al., 2024; Liu et al., 2023; Zhou et al., 2023). Pool- and router-size studies use MathBench, KOR-Bench, OCRBench, and FollowBench-zh, with a fixed held-out 10% that no router explores. The main suite has 15 text and vision benchmarks from LLMRouterBench, MMR-Bench, and RobustRouteBench (Li et al., 2026; Ma et al., 2026; SinapisAI, 2026). The API suite has ten: Omni-MATH (Gao et al., 2024), BBEH Mini (Kazemi et al., 2025), HLE (text) (Phan et al., 2026), IFBench (Pyatkin et al., 2025), AA-LCR v1.1 (Artificial Analysis, 2025), LongBench v2 (Bai et al., 2025), MMMU-Pro (Yue et al., 2025), ChartQAPro (Masry et al., 2025), Design2Code-Hard (Si et al., 2025), and LiveCodeBench-Pro (Zheng et al., 2025), the last scored by GPT-6 Luna code review.

Table 3: Closed-source model routing. Avg. is weighted by test-set size; Cost is total inference cost at official prices (USD); -: not available.

Method	Benchmark scores (%) ↑										Weighted Avg. (%) ↑	Total cost ↓ (USD)
	Omni-MATH	BBEH Mini	HLE Text	IFBench	AA-LCR v1.1	LongBench v2	MMMU-Pro	ChartQAPro	Design2Code Hard	LiveCodeBench Pro		
I. Direct models												
GPT-6.1 Sol (low)	64.32	20.49	17.77	48.00	66.00	60.18	62.39	26.79	82.36	69.15	47.23	429.79
GPT-6 Astra (low)	65.52	21.71	16.99	44.40	74.00	57.30	63.51	26.97	84.05	62.77	47.57	2157.90
Gemini 4 Argon	84.14	22.68	46.70	58.00	72.00	66.59	85.94	52.18	70.71	71.28	68.35	260.53
Claude Sonnet 5.5	84.89	30.00	33.01	38.80	62.00	63.50	77.31	23.01	73.42	55.32	59.74	257.26
GPT-6.1 Sol (max)	-	-	52.92	-	83.00	-	85.95	-	-	-	-	-
GPT-6 Astra (max)	-	-	54.68	-	80.67	-	86.88	-	-	-	-	-
II. Released and prompted routing methods												
StartLux 0.8B (Original)	79.30	21.71	34.55	51.20	74.00	56.42	76.85	51.65	77.56	68.09	62.14	196.70
JevAny-4B (Original)	79.80	30.49	20.91	43.60	64.00	61.73	64.03	26.84	76.64	58.51	54.26	201.46
Qwen3.5-4B (Prompted Selector)	76.46	40.73	30.23	65.20	76.00	56.86	68.80	45.36	76.31	69.15	59.19	75.01
III. Feedback-trained routing methods												
MLP Reward Predictor	86.27	19.76	46.55	58.00	62.00	58.41	84.76	48.34	82.91	73.40	67.90	246.87
LatentRouter (adapted)	80.93	44.88	38.11	67.60	58.00	60.40	77.83	49.61	83.16	67.02	64.67	123.31
Router-R1 (PPO)	86.78	23.66	48.92	60.00	72.00	66.81	88.29	53.80	79.70	74.47	70.53	283.26
IV. RL-trained routing methods												
StartLux 0.8B (Full FT, ours)	86.68	49.02	48.66	66.80	82.00	62.83	87.44	53.42	83.43	74.47	71.31	120.46

Baselines and metrics. We compare fixed experts, random routing, released or prompted selectors (StartLux, 2026; SimpleJev, 2026; Tran et al., 2025; Strands Labs, 2026), feedback-trained routers (Zhuang et al., 2024; Li et al., 2012), and, for the API pool, Router-R1 (Zhang et al., 2025a) adapted to a single decision and trained with PPO from the same StartLux-0.8B on the same questions. Our main router is StartLux-0.8B (full finetuning or LoRA; JevAny-style alternative in Appendix A). Avg. is request-weighted; open-weight Cost follows Eq. (2), and API Cost is token cost at list prices.

5.2 Continual Adaptation to New Tasks

Learned routers are updated stage by stage on new datasets only, and Table 2 reports their scores on all datasets seen so far.

Retention and plasticity. At the final stage, neither StartLux variant of AdaptRouter drops by more than 0.4 points on any earlier dataset, whereas LinUCB, the MLP predictor, and the latent-factor router drop by 4.4, 6.4, and 7.0 points on MATH-500. Each new dataset is learned in its stage: full finetuning lifts the released StartLux-0.8B from 39.80 to 92.20 on MATH-500 and from 21.44 to 85.84 on ChartQA. After the final stage, the two variants average 82.3–84.3 over the five datasets, on par with the fixed 9B experts (84.5 and 83.6) at 14.8–16.8% of the reasoning expert's cost; the JevAny-style router behaves similarly (Table 9).

Table 4: Pool-size study (%). NT/T: non-thinking/thinking.

Method	Math-Bench	KOR-Bench	OCR-Bench	Follow-Bench-zh	All
Qwen3.5-9B (NT)	95.3	58.0	83.1	44.3	69.3
Qwen3.5-9B (T)	96.3	76.5	81.4	61.8	77.3
Step3-VL-10B (T)	96.5	80.0	79.9	40.7	71.1
Gemma-4-12B (NT)	95.0	71.3	72.4	58.5	72.0
JevAny-0.8B (M3)	93.5	64.0	78.4	38.7	66.7
JevAny-0.8B (M5)	93.5	71.8	79.6	43.2	69.8
JevAny-0.8B (M10)	93.8	72.3	85.9	54.6	75.5

5.3 Adaptation to New Expert Pools

Since experts enter as cards, we train one router design on M3, M5, and M10 and ask whether it uses added experts and how much exploration a new pool needs.

Using added experts. For each pool, a frozen-backbone JevAny-0.8B learns only its pointer head under a matched reward and update budget. Accuracy rises with the pool, from 66.7% (M3) to 69.8% (M5) and 75.5% (M10) (Table 4), and the router puts new experts to work (Fig. 3): in M10, Gemma-4-12B receives 83% of KOR-Bench questions, and MiniCPM-V-4.5 and Step3-VL together receive 49% of OCRBench questions.

Exploration budget for a new pool. Table 5 lets the M10 router explore 1–90% of the questions with 500 updates per run, learning only from the rewards of its own decisions. A fixed 10% of the questions (whole question families) is excluded

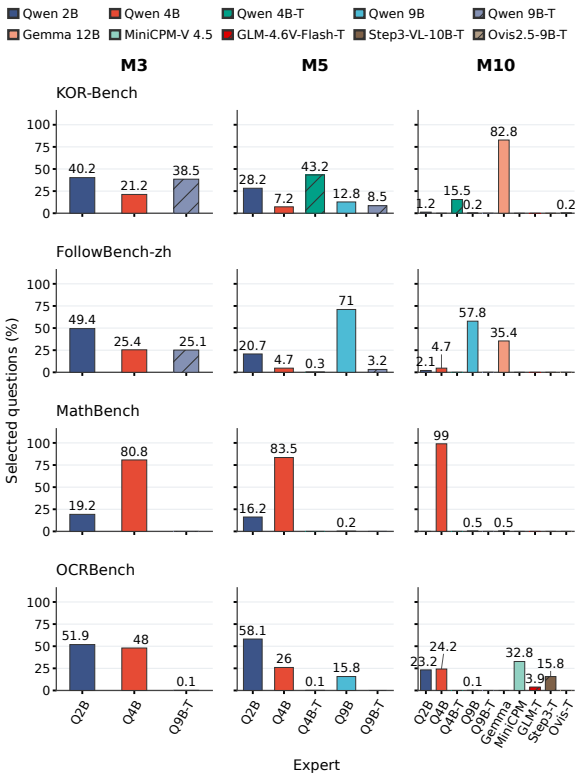


Figure 3: **Expert selection shares across datasets and pools.** Q denotes Qwen; T and hatching indicate reasoning mode.

Table 5: M10 accuracy (%) by exploration share. Held-out: the same fixed 10% of questions. Non-Qwen: selection share.

Explored	Math-Bench	KOR-Bench	OCR-Bench	Follow-Bench-zh	All	Held-out (10%)	Non-Qwen
1%	92.3	67.0	84.4	38.7	69.1	73.1	27.5
5%	91.5	76.0	81.8	47.1	72.1	77.2	33.7
10%	93.5	76.8	84.4	53.8	75.5	77.2	35.8
15%	93.8	72.3	85.9	54.6	75.5	79.1	43.6
20%	93.3	76.0	85.6	56.8	76.7	78.7	50.7
50%	93.5	76.5	86.6	59.7	78.0	78.0	62.5
60%	93.8	77.8	86.3	58.1	77.7	78.7	48.0
70%	93.3	76.8	86.5	59.0	77.8	78.4	56.8
80%	93.8	76.8	87.9	59.0	78.3	77.6	73.8
90%	93.3	77.3	87.0	58.5	77.9	77.6	58.6

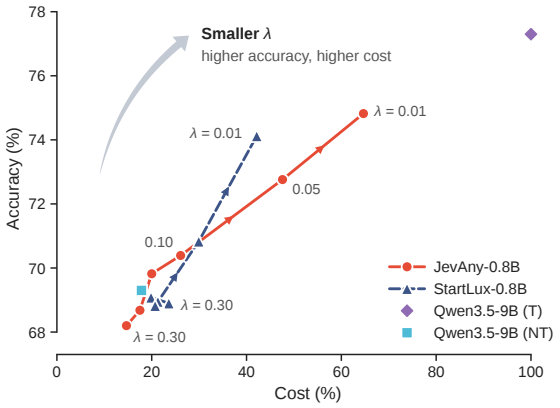


Figure 4: **Effect of the trade-off coefficient λ.** Accuracy versus cost of routers trained with different λ on the M5 expert pool. Cost is shown as a percentage of the Qwen3.5-9B (T) cost.

from every budget and serves as the held-out set. Held-out accuracy rises from 73.1% at 1% to 77.2% at 5% and then stays between 77.2% and 79.1% up to 90%: a small exploration budget suffices to learn a new pool.

5.4 Routing Quality and Cost

On the main suite (Table 1), AdaptRouter raises the released StartLux-0.8B from 44.09 to 65.08 Avg. with full finetuning (63.04 with LoRA), above the released JevAny-4B (55.24). It reaches 96.7% of the accuracy of the 9B reasoning expert at 12.56% of its cost, and comes within 0.73 points of the 9B non-reasoning expert at three quarters of its cost. Feedback-trained routers operate at a similar cost (11.86–12.74%) but reach at most 62.81; the LoRA variant is cheaper than all of them (8.81%) and more accurate.

5.5 Adapting to a Pool of API Models

Training on the API pool. Table 3 applies AdaptRouter to a new pool of five API models, none priced above GPT-6.1 Sol per token; the cards state token prices. Each training question is answered by all five experts and scored, and the router is trained toward a low-temperature softmax over their rewards $q - \lambda c/\bar{c}$ at list prices; long inputs reach it as the question, its length, and document excerpts (Appendix B).

Quality and cost. StartLux reaches the highest weighted average in the table, 71.31, which is 24.08 and 23.74 points above GPT-6.1 Sol and GPT-6 Astra at low reasoning effort (47.23 and 47.57) at 28% and 6% of their cost; it scores higher than both on nine of the ten benchmarks. Against Gemini 4 Argon (68.35, $260.53) and Claude Sonnet 5.5 (59.74, $257.26) used alone, it is higher on nine benchmarks each at less than half of their cost. Among routing baselines, Router-R1, trained from the same decision model on the same questions, comes closest (70.53) at 2.35 times the cost, and only the Prompted Selector is cheaper ($75.01), at 12.12 points lower.

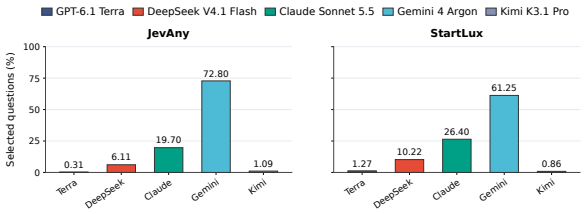


Figure 5: Model selection shares across the five API models for JevAny (left; Table 10) and StartLux (right; Table 3).

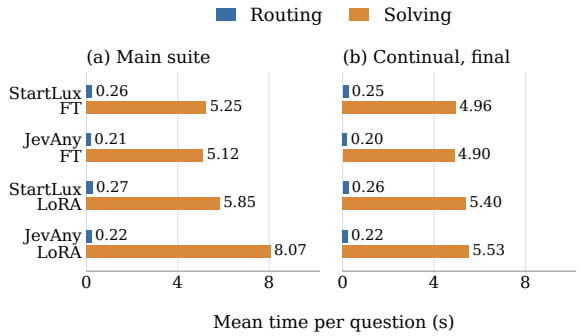


Figure 6: **Routing and expert execution time.** Mean time per question on the main suite (Table 1) and the final continual stage (Table 2).

Where the savings come from. Although StartLux sends nearly nine in ten questions to Gemini 4 Argon or Claude Sonnet 5.5 (Fig. 5), it costs less than half as much as either alone, because cost concentrates on long-context questions. LongBench v2 has only 452 questions (4.36%), yet costs $112.28 with Gemini 4 Argon; StartLux sends almost all of them to DeepSeek V4.1 Flash for $9.54, saving $102.74 on this benchmark alone.

5.6 Analysis

Routing time and λ. Mean routing takes 0.20–0.27 seconds per question, while the selected experts take 4.90–8.07 GPU seconds (Fig. 6). As λ decreases from 0.30 to 0.01, StartLux rises from 68.9% to 74.1% accuracy while its cost grows from 23.6% to 42.2% of Qwen3.5-9B (T) (Fig. 4); details are in Appendix A.4.

Router scale. Table 6 trains StartLux Instant routers and cold-start JevAny-style routers at 0.8B, 4B, and 9B on M5 with the same data, reward, and LoRA budget. StartLux reaches 68.95, 69.17, and 67.06% All, and JevAny-style 70.39, 69.65, and 67.76%; in both families a larger router brings no reliable gain, and the 0.8B router is the fastest.

Table 6: Router scale on M5 (%). Mean/p95: router latency per question incl. preprocessing. Order consist.: pairwise agreement of selected experts across five cyclic candidate orders, measuring the stability of the selection.

Size	Math-Bench	KOR-Bench	OCR-Bench	Follow-Bench-zh	All	Mean (ms)	p95 (ms)	Order consist. (%)
StartLux Instant (LoRA)								
0.8B	93.00	66.75	80.25	42.79	68.95	262.6	343.1	43.19
4B	92.75	61.75	79.50	47.50	69.17	382.3	508.0	43.66
9B	92.50	59.50	78.13	43.53	67.06	385.3	498.8	56.45
JevAny-style (LoRA)								
0.8B	92.50	71.50	78.63	47.06	70.39	230.8	305.2	17.49
4B	92.25	69.50	80.13	44.12	69.65	298.1	410.3	22.15
9B	92.00	60.50	80.63	42.65	67.76	292.3	394.3	25.55

Table 7: Ablations of AdaptRouter (StartLux-0.8B Full FT; %). Small: selection share of the small non-reasoning experts (2B/4B).

Variant	Math-Bench	KOR-Bench	OCR-Bench	Follow-Bench-zh	All	Small
AdaptRouter (Full FT)	96.20	58.40	83.00	46.41	70.22	32.80
(a) w/o quality-first adv.	95.20	57.60	82.00	45.90	69.40	57.35
(b) w/o small-expert term	95.80	57.20	82.44	47.05	69.93	5.26
(c) w/o reasoning-gain term	95.80	58.00	82.89	46.79	70.15	24.85
(d) w/o visual input	95.20	57.80	82.33	45.38	69.40	42.84

5.7 Ablation Study

Table 7 removes one component at a time; the full model has the best All score, and every removal lowers it. (a) Replacing the quality-first advantage with A_i^R lets cost outweigh quality: the small-expert share rises from 32.80% to 57.35% and accuracy drops on all four benchmarks (All 69.40), as Proposition 1 predicts. (b) Without the small-expert term, small experts are almost never selected (5.26%; All 69.93). (c) Without the reasoning-gain term, All decreases to 70.15. (d) Blanking the router's images lowers accuracy on all four benchmarks and All to 69.40.

6 Conclusion

We introduced AdaptRouter, which casts model routing as continual policy learning over cards that describe the current experts. It learns new tasks without replay where feedback-trained routers forget, puts added experts to use from a small exploration budget, and keeps quality ahead of cost. On open-weight and API pools, it routes accurately at a fraction of the cost of single models. A natural next step is to update a single router continually as experts are added, retired, and repriced.

Limitations

AdaptRouter routes each question to one frozen expert and does not cover collaboration or cascades. Each new task or pool needs an automatic scorer or judge and an exploration budget, and our continual sequence keeps the pool fixed; updating one router continually as experts change is left to future work. Our four-benchmark studies reuse questions analyzed during development, and our costs omit batching and end-to-end latency.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. 2024. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms. arXiv preprint arXiv:2402.14740.

Artificial Analysis. 2025. Artificial Analysis Long Context Reasoning (AA-LCR). Hugging Face dataset `ArtificialAnalysis/AA-LCR`.

Yushi Bai, Shangqing Tu, Jiajie Zhang, Hao Peng, Xiaozhi Wang, Xin Lv, Shulin Cao, Jiazheng Xu, Lei Hou, Yuxiao Dong, et al. 2025. LongBench v2: Towards deeper understanding and reasoning on realistic long-context multitasks. arXiv preprint arXiv:2412.15204.

Cheng Chen, Junchen Zhu, Xu Luo, Hengtao Shen, Lianli Gao, and Jingkuan Song. 2024a. Coin: A benchmark of continual instruction tuning for multimodel large language model. arXiv preprint arXiv:2403.08350.

Liang Chen, Haozhe Zhao, Tianyu Liu, Shuai Bai, Junyang Lin, Chang Zhou, and Baobao Chang. 2024b. An image is worth 1/2 tokens after layer 2: Plug-and-play inference acceleration for large vision-language models. arXiv preprint arXiv:2403.06764.

Lingjiao Chen, Matei Zaharia, and James Zou. 2023. Frugalgpt: How to use large language models while reducing cost and improving performance. arXiv preprint arXiv:2305.05176.

Shuhao Chen, Weisen Jiang, Baijiong Lin, James T. Kwok, and Yu Zhang. 2024c. Routerdc: Query-based router by dual contrastive learning for assembling large language models. arXiv preprint arXiv:2409.19886.

DeepSeek-AI, Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, et al. 2026. Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. arXiv preprint arXiv:2501.12948.

Jasper Dekoninck, Maximilian Baader, and Martin Vechev. 2025. A unified approach to routing and cascading for LLMs. In *Proceedings of the 42nd International Conference on Machine Learning*, pages 12987–13010.

Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Ruhle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. 2024. Hybrid llm: Cost-efficient and quality-aware query routing. arXiv preprint arXiv:2404.14618.

Gongfan Fang, Xinyin Ma, and Xinchao Wang. 2025. Thinkless: Llm learns when to think. arXiv preprint arXiv:2505.13379.

Tao Feng, Yanzhen Shen, and Jiaxuan You. 2025. Graphrouter: A graph-based router for llm selections. arXiv preprint arXiv:2410.03834.

Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, et al. 2024. Omni-MATH: A universal olympiad level mathematic benchmark for large language models. arXiv preprint arXiv:2410.07985.

Yizeng Han, Gao Huang, Shiji Song, Le Yang, Honghui Wang, and Yulin Wang. 2021. Dynamic neural networks: A survey. arXiv preprint arXiv:2102.04906.

Jinghan He, Haiyun Guo, Ming Tang, and Jinqiao Wang. 2023. Continual instruction tuning for large multimodal models. arXiv preprint arXiv:2311.16206.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. Lora: Low-rank adaptation of large language models. arXiv preprint arXiv:2106.09685.

Qitian Jason Hu, Jacob Bieker, Xiuyu Li, Nan Jiang, Benjamin Keigwin, Gaurav Ranganath, Kurt Keutzer, and Shriyash Kaustubh Upadhyay. 2024. Routerbench: A benchmark for multi-llm routing system. arXiv preprint arXiv:2403.12031.

Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 14165–14178. Association for Computational Linguistics.

Mehran Kazemi, Bahare Fatemi, Hritik Bansal, John Palowitch, Chrysovalantis Anastasiou, Sanket Vaibhav Mehta, Lalit K. Jain, Virginia Aglietti, Disha Jindal, Peter Chen, et al. 2025. BIG-Bench extra hard. arXiv preprint arXiv:2502.19187.

James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, et al. 2017. Overcoming catastrophic forgetting in neural networks. arXiv preprint arXiv:1612.00796.

Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient memory management for large language model serving with pagedattention. arXiv preprint arXiv:2309.06180.

Hao Li, Yiqun Zhang, Zhaoyan Guo, Chenxu Wang, Shengji Tang, Qiaosheng Zhang, Yang Chen, Biqing Qi, Peng Ye, Lei Bai, et al. 2026. Llmrouterbench: A massive benchmark and unified framework for llm routing. arXiv preprint arXiv:2601.07206.

Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. 2012. A contextual-bandit approach to personalized news article recommendation. arXiv preprint arXiv:1003.0146.

Zhizhong Li and Derek Hoiem. 2017. Learning without forgetting. arXiv preprint arXiv:1606.09282.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's verify step by step. arXiv preprint arXiv:2305.20050.

Bin Lin, Zhenyu Tang, Yang Ye, Jinfa Huang, Junwu Zhang, Yatian Pang, Peng Jin, Munan Ning, Jiebo Luo, and Li Yuan. 2024. Moe-llava: Mixture of experts for large vision-language models. arXiv preprint arXiv:2401.15947.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. arXiv preprint arXiv:2305.01210.

Ziyu Liu, Zeyi Sun, Yuhang Zang, Xiaoyi Dong, Yuhang Cao, Haodong Duan, Dahua Lin, and Jiaqi Wang. 2025. Visual-rft: Visual reinforcement fine-tuning. arXiv preprint arXiv:2503.01785.

Keming Lu, Hongyi Yuan, Runji Lin, Junyang Lin, Zheng Yuan, Chang Zhou, and Jingren Zhou. 2024. Routing to the expert: Efficient reward-guided ensemble of large language models. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 1964–1974. Association for Computational Linguistics.

Haoxuan Ma, Guannan Lai, and Han-Jia Ye. 2026. Mmr-bench: A comprehensive benchmark for multimodal llm routing. arXiv preprint arXiv:2601.17814.

Ahmed Masry, Mohammed Saidul Islam, Mahir Ahmed, Aayush Bajaj, Firoz Kabir, Aaryaman Kartha, Md Tahmid Rahman Laskar, Mizanur Rahman, Shadikur Rahman, Mehrad Shahmohammadi, et al. 2025. ChartQAPro: A more diverse and challenging benchmark for chart question answering. arXiv preprint arXiv:2504.05506.

Ahmed Masry, Do Xuan Long, Jia Qing Tan, Shafiq Joty, and Enamul Hoque. 2022. Chartqa: A benchmark for question answering about charts with visual and logical reasoning. arXiv preprint arXiv:2203.10244.

Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M Waleed Kadous, and Ion Stoica. 2025. Routellm: Learning to route llms with preference data. arXiv preprint arXiv:2406.18665.

Long Phan, Alice Gatti, Ziwen Han, Nathaniel Li, Josephina Hu, Hugh Zhang, Chen Bo Calvin Zhang, Mohamed Shaaban, John Ling, Sean Shi, et al. 2026. Humanity's last exam. arXiv preprint arXiv:2501.14249.

Reiner Pope, Sholto Douglas, Aakanksha Chowdhery, Jacob Devlin, James Bradbury, Anselm Levskaya, Jonathan Heek, Kefan Xiao, Shivani Agrawal, and Jeff Dean. 2023. Efficiently scaling transformer inference. In *Proceedings of Machine Learning and Systems*.

Valentina Pyatkin, Saumya Malik, Victoria Graf, Hamish Ivison, Shengyi Huang, Pradeep Dasigi, Nathan Lambert, and Hannaneh Hajishirzi. 2025. Generalizing verifiable instruction following. arXiv preprint arXiv:2507.02833.

Qwen Team. 2026. Qwen3.5 model family. Hugging Face models `Qwen/Qwen3.5-*`.

David Raposo, Sam Ritter, Blake Richards, Timothy Lillicrap, Peter Conway Humphreys, and Adam Santoro. 2024. Mixture-of-depths: Dynamically allocating compute in transformer-based language models. arXiv preprint arXiv:2404.02258.

Sylvestre-Alvise Rebuffi, Alexander Kolesnikov, Georg Sperl, and Christoph H. Lampert. 2017. icarl: Incremental classifier and representation learning. arXiv preprint arXiv:1611.07725.

David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. 2019. Experience replay for continual learning. arXiv preprint arXiv:1811.11682.

Yuzhang Shang, Mu Cai, Bingxin Xu, Yong Jae Lee, and Yan Yan. 2026. Llava-prumerge: Adaptive token reduction for efficient large multimodal models. arXiv preprint arXiv:2403.15388.

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, et al. 2024. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. arXiv preprint arXiv:2402.03300.

Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. arXiv preprint arXiv:1701.06538.

Haozhan Shen, Peng Liu, Jingcheng Li, Chunxin Fang, Yibo Ma, Jiajia Liao, Qiaoli Shen, Zilun Zhang, Kangjia Zhao, Qianqian Zhang, et al. 2025. Vlm-r1: A stable and generalizable r1-style large vision-language model. arXiv preprint arXiv:2504.07615.

Chenglei Si, Yanzhe Zhang, Ryan Li, Zhengyuan Yang, Ruibo Liu, and Diyi Yang. 2025. Design2Code: Benchmarking multimodal code generation for automated front-end engineering. arXiv preprint arXiv:2403.03163.

SimpleJev. 2026. JevAny: pointer-head decision model. Hugging Face model SimpleJev/JevAny-Qwen3.5-4B-LoRA.

SinapisAI. 2026. RobustRouteBench. Hugging Face dataset SinapisAI/RobustRouteBench.

Vikranth Srivatsa, Zijian He, Reyna Abhyankar, Dongming Li, and Yiying Zhang. 2024. Preble: Efficient distributed prompt scheduling for llm serving. arXiv preprint arXiv:2407.00023.

StartLux. 2026. StartLux-Decision v1.1 Instant. Hugging Face models startlux-models/StartLux-Decision-v1.1-*-Instant.

Strands Labs. 2026. Strands decider. Hugging Face model StrandsAgents/strands-decider-2B-qwen3.5-v1-2610.

Co Tran, Salman Paracha, Adil Hafeez, and Shuguang Chen. 2025. Arch-router: Aligning llm routing with human preferences. arXiv preprint arXiv:2506.16655.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, et al. 2024. Mmlu-pro: A more robust and challenging multi-task language understanding benchmark. arXiv preprint arXiv:2406.01574.

Zichen Wen, Yifeng Gao, Weijia Li, Conghui He, and Linfeng Zhang. 2025. Token pruning in multimodal large language models: Are we solving the right problem? arXiv preprint arXiv:2502.11501.

Ronald J. Williams. 1992. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8(3–4):229–256.

Senqiao Yang, Yukang Chen, Zhuotao Tian, Chengyao Wang, Jingyao Li, Bei Yu, and Jiaya Jia. 2026. Visionzip: Longer is better but not necessary in vision language models. arXiv preprint arXiv:2412.04467.

Tao Yu, Yifei Qu, Zhiqing Cui, Pengfei Zhou, Zhongtian Luo, Yujia Yang, Shenghua Chai, Haopeng Jin, Zhenghao Zhang, Xinming Wang, et al. 2026. SCOPE-Router: Cost-aware open-set VLM routing for execution-oriented tasks. arXiv preprint arXiv:2608.12127.

Murong Yue, Jie Zhao, Min Zhang, Liang Du, and Ziyu Yao. 2024. Large language model cascades with mixture of thoughts representations for cost-efficient reasoning. arXiv preprint arXiv:2310.03094.

Xiang Yue, Tianyu Zheng, Yuansheng Ni, Yubo Wang, Kai Zhang, Shengbang Tong, Yuxuan Sun, Botao Yu, Ge Zhang, Huan Sun, et al. 2025. MMMU-Pro: A more robust multi-discipline multimodal understanding benchmark. arXiv preprint arXiv:2409.02813.

Haozhen Zhang, Tao Feng, and Jiaxuan You. 2025a. Router-r1: Teaching llms multi-round routing and aggregation via reinforcement learning. arXiv preprint arXiv:2506.09033.

Jiajie Zhang, Nianyi Lin, Lei Hou, Ling Feng, and Juanzi Li. 2025b. Adaptthink: Reasoning models can learn when to think. arXiv preprint arXiv:2505.13417.

Yuan Zhang, Chun-Kai Fan, Junpeng Ma, Wenzhao Zheng, Tao Huang, Kuan Cheng, Denis Gudovskiy, Tomoyuki Okuno, Yohei Nakata, Kurt Keutzer, et al. 2025c. Sparsevlm: Visual token sparsification for efficient vision-language model inference. arXiv preprint arXiv:2410.04417.

Zihan Zheng, Zerui Cheng, Zeyu Shen, Shang Zhou, Kaiyuan Liu, Hansen He, Dongruixuan Li, Stanley Wei, Hangyi Hao, Jianzhu Yao, et al. 2025. LiveCodeBench Pro: How do olympiad medalists judge LLMs in competitive programming? arXiv preprint arXiv:2506.11928.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-following evaluation for large language models. arXiv preprint arXiv:2311.07911.

Pengfei Zhou, Yixing Ma, Hexin Wang, Weidong Tang, Zhenglin Wan, Wangbo Zhao, and Yang You. 2026a. Route anything: A study of model and policy routing for general-purpose AI systems. arXiv preprint arXiv:2611.05275.

Pengfei Zhou, Zhiwei Tang, Yixing Ma, Jiasheng Tang, Yizeng Han, Zhenglin Wan, Fanqing Meng, Wei Wang, Bohan Zhuang, Wangbo Zhao, and Yang You. 2026b. Agent-as-a-router: Agentic model routing for coding tasks. arXiv preprint arXiv:2606.22902.

Richard Zhuang, Tianhao Wu, Zhaojin Wen, Andrew Li, Jiantao Jiao, and Kannan Ramchandran. 2024. Embedllm: Learning compact representations of large language models. arXiv preprint arXiv:2410.02223.

A Alternative Router

We use StartLux-0.8B in the main comparisons because its full-finetuning variant has the higher average on the main suite (65.08 versus 63.04) and the higher final-stage continual average (84.3 versus 83.9). To show that the method also works with a different decision architecture, we report all JevAny-style results below alongside the StartLux reference rows.

Pointer-head parameterization. JevAny (SimpleJev, 2026) encodes the request jointly with its candidates and scores each candidate with a pointer head. Since the released JevAny model is 4B, we construct the 0.8B router from its published architecture on Qwen3.5-0.8B and initialize the query projection to zero, yielding a uniform initial policy with non-vanishing gradients. It is trained without the decision pretraining of the released JevAny-4B. Both router designs use the same reward and optimizer and are trained with full finetuning or LoRA for the main suite.

A.1 Main-suite Results

Table 8 compares both designs on all 15 benchmarks under the protocol of Table 1. JevAny-0.8B reaches 63.04 with full finetuning and 62.01 with LoRA; the former is 7.80 points above the released JevAny-4B (55.24), despite the smaller backbone. StartLux has the higher average in both training modes, while the alternative remains competitive across text and vision tasks.

A.2 Continual Routing

Table 9 retains all four training variants under the protocol of Table 2. No variant loses more than 0.4 points on an earlier dataset relative to its best earlier stage. Both JevAny variants reach a final five-dataset average of 83.9, compared with 84.3 and 82.3 for StartLux full finetuning and LoRA.

A.3 Routing API Models

Table 10 compares both routers under the protocol of Table 3. Here StartLux has the higher weighted average by 0.45 points (71.31 versus 70.86). Both routers exceed both low-effort single-model references on nine of the ten benchmarks; Fig. 5 shows their different expert-selection shares.

A.4 Trade-off across Router Designs

Decreasing λ from 0.30 to 0.01 raises JevAny-0.8B from 68.2% to 74.8% accuracy as its cost grows from 14.7% to 64.7% of Qwen3.5-9B (T) (Fig. 4). For $\lambda \geq 0.10$, both routers stay within 68.2–70.4% accuracy and 14.7–26.1% cost; the gains at smaller λ concentrate on FollowBench-zh and KOR-Bench, where JevAny-0.8B shifts its 9B share from the non-reasoning to the reasoning mode.

B API Router Training

Both API routers (StartLux-0.8B and the JevAny-style alternative) are fully finetuned on the API training questions, with the ten datasets weighted equally. With expert qualities q_k and list-price costs c_k, the rewards are $u_k = q_k - \lambda c_k/\bar{c}$, where $\lambda = 0.02$ and $\bar{c}$ is the mean training expert cost averaged over datasets; the target is $y = \mathrm{softmax}(u/\tau)$ with $\tau = 0.05$, and with $p_k = \pi_\theta(e_k \mid x, \mathcal{E})$ the loss is

$$-\textstyle\sum_k y_k \log p_k + 0.1\left(\max_k u_k - \sum_k p_k u_k\right), \quad (8)$$

whose second term is the expected regret. Training runs three epochs with 16 questions per update, AdamW (learning rate 10^{-6}), and gradient clipping at 1.0; the candidate order is shuffled per question, and the router sees up to 16 distinct images.

C Hyperparameters

Table 11 lists the two training recipes; notation follows Section 4. The main recipe trains StartLux-0.8B for Tables 1 and 7, and each continual stage of Table 2 applies it to that stage's new questions. The four-benchmark recipe trains a JevAny-0.8B pointer head on a frozen backbone for Tables 4 and 5; on M3 it omits both reasoning bonuses. The full model in Table 7 was trained on one GPU and each ablation variant on eight. ChartQA uses the revised scoring protocol; all other datasets use native scores.

Table 11: Training hyperparameters.

	Main	Four-benchmark
Router	StartLux-0.8B	JevAny-0.8B
Trainable parameters	all	pointer head
Learning rate	10^{-6}	10^{-4}, linear decay
Training questions	1,700	400
Epochs / updates	1 / 107	20 / 500
Questions per update	16	16
Attempts G	8	8
Exploration ϵ	0.3	0.3
Cost weight λ	0.10	0.15
Small-expert bonus β_s	0.15	0.10
Reasoning bonus β_t	16	16
4B (T) unique-correct bonus	–	4
Advantage bound ρ	0.25	0.25
Entropy coefficient η	0.05	0.05
Uniform-KL coefficient	–	0.1
Seed	37	37

Table 8: Router-design comparison on the main suite. Native scores and request-weighted Avg. (%); Cost follows Table 1.

Method	LLMRouterBench					MMR-Bench V2						RobustRouteBench				Avg.	Cost
	Acc. ↑ MathBench	Acc. ↑ KOR-Bench	Acc. ↑ SimpleQA	Acc. ↑ FinQA	Acc. ↑ MELD	Acc. ↑ RealWorldQA	Acc. ↑ OCRBench	Acc. ↑ CharXiv	Acc. ↑ MathVerse	Acc. ↑ BLINK	aAcc. ↑ HallusionBench	Acc. ↑ HellaSwag	HSR ↑ FollowBench-zh	Acc. ↑ C-Eval	Acc. ↑ AI2D		
StartLux 0.8B (Full FT, ours)	96.20	58.40	5.80	37.60	60.20	69.24	83.00	57.70	72.67	61.20	72.55	89.20	46.41	81.00	82.00	65.08	12.56%
StartLux 0.8B (LoRA, ours)	94.80	59.00	5.20	36.40	51.00	65.45	80.78	56.10	69.09	62.20	71.94	84.40	45.00	80.00	82.40	63.04	8.81%
JevAny-0.8B (Full FT, ours)	95.20	58.20	4.20	36.00	55.80	67.67	81.33	54.70	66.92	61.20	71.14	85.80	44.49	80.20	82.20	63.04	10.88%
JevAny-0.8B (LoRA, ours)	95.40	55.60	4.40	33.80	53.40	66.49	81.22	53.30	71.01	59.40	71.94	78.60	43.46	76.00	82.40	62.01	12.14%

Table 9: Continual routing with both router designs (%). Stage-4 parentheses: change from each dataset's first stage; Cost as in Table 2.

Method	Stage 1		Stage 2			Stage 3				Stage 4 (change vs. first)					S4 cost
	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ Human Eval+	Acc. ↑ MATH-500	Acc. ↑ Chart QA	Acc. ↑ MMLU-Pro	Acc. ↑ Human Eval+	Acc. ↑ IF Eval	
StartLux 0.8B (Full FT, ours)	92.20	85.84	91.80	85.44	74.00	92.00	85.52	73.80	84.10	92.20 (+0.00)	86.02 (+0.18)	74.40 (+0.40)	84.32 (+0.22)	84.66	16.83%
StartLux 0.8B (LoRA, ours)	84.60	83.38	89.40	84.59	71.80	90.60	85.52	73.40	78.66	90.80 (+6.20)	85.68 (+2.30)	73.00 (+1.20)	78.66 (+0.00)	83.36	14.77%
JevAny-0.8B (Full FT, ours)	83.20	83.58	90.20	84.91	71.80	91.00	84.63	72.80	80.49	92.00 (+8.80)	85.84 (+2.26)	74.40 (+2.60)	81.71 (+1.22)	83.92	16.73%
JevAny-0.8B (LoRA, ours)	84.00	83.30	90.00	84.83	72.60	91.20	84.95	75.00	83.54	91.20 (+7.20)	84.79 (+1.49)	74.60 (+2.00)	83.54 (+0.00)	83.92	15.62%

Table 10: API-model routing with both router designs. Native scores and request-weighted Avg. (%); total selected-expert token cost at official prices (USD).

Method	Benchmark scores (%) ↑										Weighted Avg. (%) ↑	Total cost ↓ (USD)
	Omni-MATH	BBEH Mini	HLE Text	IFBench	AA-LCR v1.1	LongBench v2	MMMU-Pro	ChartQAPro	Design2Code Hard	LiveCodeBench Pro		
StartLux 0.8B (Full FT, ours)	86.68	49.02	48.66	66.80	82.00	62.83	87.44	53.42	83.43	74.47	71.31	120.46
JevAny-0.8B (Full FT, ours)	86.60	33.90	48.92	64.40	82.00	62.83	88.36	53.75	83.43	74.47	70.86	134.30